

Module 9) Python DB and Framework

1. HTML in Python

Theory:

□ Introduction to embedding HTML within Python using web frameworks like Django or Flask.

Embedding HTML within Python means creating web pages using Python code with the help of web frameworks such as **Django** and **Flask**. These frameworks allow developers to combine Python logic with HTML to build dynamic and interactive websites.

□ Generating dynamic HTML content using Django templates.

In **Django**, dynamic HTML content is generated using the template system. Django templates allow developers to insert variables, loops, and conditions inside HTML files so that data from the backend can be displayed dynamically on the web page.

2. CSS in Python

Theory:

□ Integrating CSS with Django templates.

In **Django**, CSS is integrated with templates to style the HTML web pages. CSS files are stored in the **static folder** of the Django project. These files are then linked to the HTML templates using the <link> tag. This allows developers to apply styles such as colors, fonts, layouts, and spacing to the web pages.

□ How to serve static files (like CSS, JavaScript) in Django.

In **Django**, static files such as CSS, JavaScript, and images are stored in a **static folder** inside the project or app directory. Django uses the **STATIC_URL** and **STATICFILES_DIRS** settings in settings.py to locate these files. The {% load static %} template tag is used in HTML templates to link static files, allowing them to be properly served and displayed on web pages.

3. JavaScript with Python

Theory:

□ Using JavaScript for client-side interactivity in Django templates.

In **Django**, JavaScript is used in templates to provide client-side interactivity. It allows actions such as form validation, dynamic content updates, pop-up messages, and animations without reloading the webpage. This improves the user experience and makes web applications more interactive.

❑ Linking external or internal JavaScript files in Django.

In **Django**, JavaScript files can be linked to HTML templates in two ways.

1. **External JavaScript:** Files are stored in the **static folder** and included using the `<script src="">` tag.
2. **Internal JavaScript:** JavaScript code is written directly inside the `<script>` tag within the HTML template.

4. Django Introduction

Theory:

❑ Overview of Django: Web development framework.

Django is a high-level Python web development framework used to build secure, scalable, and maintainable web applications. It follows the **Model-View-Template (MVT)** architecture and provides many built-in features such as authentication, admin panel, and database management, which help developers build web applications quickly.

❑ Advantages of Django (e.g., scalability, security).

1. **Scalability:** It can handle large and high-traffic web applications.
2. **Security:** Provides protection against common attacks such as SQL injection, cross-site scripting, and clickjacking.
3. **Rapid Development:** Built-in tools help developers build applications quickly.
4. **Admin Panel:** Provides a ready-to-use admin interface to manage data.
5. **ORM Support:** Allows database operations using Python code instead of SQL.

❑ Django vs. Flask comparison: Which to choose and why.

Django	Flask
Full-featured web framework	Lightweight micro framework
Many built-in tools available	Requires external libraries

Suitable for large and complex projects	Suitable for small and simple applications
Provides built-in admin panel and ORM	Minimal built-in features

5. Virtual Environment

Theory:

□ Understanding the importance of a virtual environment in Python projects.

A virtual environment in Python is an isolated workspace that allows developers to install project-specific libraries without affecting the global Python installation. Tools like **venv** help create separate environments for different projects.

The importance of a virtual environment includes:

1. It keeps project dependencies isolated.
2. It prevents conflicts between different package versions.
3. It helps manage libraries for multiple projects easily.
4. It improves project organization and portability.

6. Project and App Creation

Theory:

□ Steps to create a Django project and individual apps within the project.

Steps:

1. Install Django using the command:
`pip install django`
2. Create a new Django project:
`django-admin startproject projectname`
3. Move into the project directory:
`cd projectname`
4. Create a new app inside the project:
`python manage.py startapp appname`
5. Add the app to the **INSTALLED_APPS** list in the settings.py file.

6. Run the development server using:
`python manage.py runserver`

7. MVT Pattern Architecture

Theory:

□ Django's MVT (Model-View-Template) architecture and how it handles request-response cycles.

Django follows the **MVT (Model-View-Template)** architecture to separate application logic into different components.

Components:

1. **Model:**
Handles the database structure and manages data.
2. **View:**
Contains the business logic and processes user requests.
3. **Template:**
Responsible for the presentation layer using HTML to display data.

8. Django Admin Panel

Theory:

□ Introduction to Django's built-in admin panel.

The **Django Admin Panel** is a built-in feature that provides a graphical interface to manage application data. It allows developers and administrators to add, update, and delete records in the database without writing SQL queries.

The admin panel is automatically generated by Django and can be accessed through a web browser after creating a **superuser**. It is commonly used to manage users, models, and other application data easily.

9. URL Patterns and Template Integration

Theory:

□ Setting up URL patterns in `urls.py` for routing requests to views.

In **Django**, the `urls.py` file is used to define **URL patterns** that map web addresses (URLs) to specific view functions. When a user sends a request through the browser, Django checks the URL patterns in `urls.py` and routes the request to the corresponding **view**, which processes the request.

10. Form Validation using JavaScript

Theory:

□ Using JavaScript for front-end form validation.

JavaScript is used for front-end form validation to check user input before the form is submitted to the server. It verifies fields such as email, phone number, and required inputs on the client side. This helps prevent incorrect data submission, improves user experience, and reduces server load by catching errors early.

11. Django Database Connectivity (MySQL or SQLite)

Theory:

□ Connecting Django to a database (SQLite or MySQL).

In **Django**, a database is connected through the `settings.py` file. Django supports databases such as **SQLite** and **MySQL**. The database configuration is defined in the **DATABASES** setting, where details like database name, user, password, and host are specified. After configuration, Django uses its ORM to interact with the database for storing and retrieving data.

12. ORM and QuerySets

Theory:

□ Understanding Django's ORM and how QuerySets are used to interact with the database.

In **Django**, **ORM (Object Relational Mapper)** allows developers to interact with the database using Python code instead of writing SQL queries. It converts Python objects into database tables and queries.

QuerySets are collections of database queries used to retrieve, filter, and manipulate data from the database. They allow operations like creating, reading, updating, and deleting records in a simple and efficient way.

Theory:

□ Using Django's built-in form handling.

Django provides a built-in form system that helps developers create and process forms easily. It automatically handles form generation, validation, and data processing. Django forms help reduce code and ensure that user input is properly validated before storing it in the database.

□ Implementing Django's authentication system (sign up, login, logout, password management).

Django includes a built-in authentication system that manages user accounts and security. It provides features such as **user registration (sign up), login, logout, and password management**. This system ensures secure user authentication and allows developers to easily manage user access to the web application.

14. CRUD Operations using AJAX

Theory:

□ Using AJAX for making asynchronous requests to the server without reloading the page.

AJAX (Asynchronous JavaScript and XML) is used to send and receive data from the server without reloading the entire webpage. In **Django** applications, AJAX allows web pages to update specific parts of the page dynamically. This improves user experience by making the application faster and more interactive.

GitHub Project Deployment

Theory:

□ Steps to push a Django project to GitHub.

- Install and initialize **Git** in the project folder using the command `git init`.
- Add all project files to Git using `git add ..`

- Commit the files with a message using `git commit -m "Initial commit"`.
- Create a new repository on GitHub.
- Connect the local project to the GitHub repository using `git remote add origin repository_link`.
- Push the project to GitHub using `git push -u origin main`.